

Meaning becomes coordinates, and you get to walk around in them

Shown live at the front, then yours to explore. Text becomes token IDs, IDs become vectors that are loaded rather than trained, and nearness in space starts to look like nearness in meaning.

Live, then explored

No hand-in

Pretrained vectors

Offline fallback included

Before you start

A · text becomes numbers

B · embeddings: meaning as...

Explore — five minutes...

Before you start

Run the notebook top to bottom. With internet access, set `USE_PRETRAINED = True` to load real GloVe vectors; without it, the small offline set in `data/` makes every cell run.



By the end of this session you can

- Turn a sentence into tokens and token IDs.
- Compute cosine similarity and nearest neighbours.
- Separate what the geometry returned from your interpretation of it.

PART A

text becomes numbers



A model never receives words, only integers. The simplest tokenizer splits on spaces.

in the notebook

PYTHON

```
text = "Grand-Bassam students study artificial intelligence."
tokens = text.lower().replace(".", "").split()
vocab = {tok: i for i, tok in enumerate(sorted(set(tokens)))}
ids = [vocab[t] for t in tokens]
print("tokens:", tokens)
print("IDs:   ", ids)
```

what it should print

OUTPUT

```
tokens: ['grand-bassam', 'students', 'study', 'artificial', 'intelligence']
IDs:    [1, 3, 4, 0, 2]
```

Real models use **subword** tokenizers, so an unseen word breaks into known pieces. With `transformers` installed and internet access, this shows how GPT-2 splits four awkward strings:

in the notebook

PYTHON

```
RUN_TOKENIZER = False
if RUN_TOKENIZER:
    from transformers import AutoTokenizer
    tok = AutoTokenizer.from_pretrained("distilgpt2")
    for s in ["unbelievable", "AI-driven", "Grand-Bassam", "😄"]:
        pieces = [tok.decode([i]) for i in tok.encode(s)]
        print(f"{s:<14} → {pieces}")
```

PART B

embeddings: meaning as coordinates

B



⚠ **About the offline vectors.** `data/mini_embeddings_fallback.npz` holds 34 **synthetic teaching vectors**, built by hand on eight readable axes (royalty, gender, health, animal, school, place, action, size) with a little noise. They make the mechanics runnable offline. They are *not* learned from text, so they will look tidier than real embeddings — which is itself worth noticing.

in the notebook

PYTHON

```
import numpy as np

USE_PRETRAINED = False

def cosine(a, b):
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-12))

if USE_PRETRAINED:
    import gensim.downloader as api
    emb = api.load("glove-wiki-gigaword-50")
    vector = lambda w: emb[w]
    nearest = lambda v, topn=5: emb.similar_by_vector(v, topn=topn)
else:
    data = np.load("data/mini_embeddings_fallback.npz", allow_pickle=True)
    words, matrix = [str(w) for w in data["words"]], data["vectors"]
    index = {w: i for i, w in enumerate(words)}
    vector = lambda w: matrix[index[w]]
    def nearest(v, topn=5):
        scored = sorted(((w, cosine(v, vector(w))) for w in words), key=lambda s: -s[1])
        return [(w, round(s, 3)) for w, s in scored[:topn]]

print("nearest to clinic:", nearest(vector("clinic")))
```

what it should print

OUTPUT

```
nearest to clinic: [('clinic', 1.0), ('pharmacy', 0.95), ('hospital', 0.924), ('beach',
0.784), ('grand-bassam', 0.703)]
```

SESSION 24

Explore — five minutes each, with your neighbour

1. The neighbours of `doctor`, `student` and `dog`. Which neighbour surprised you?
2. The analogy `king - man + woman`. Does the top answer include one of the three words you started from? What does that tell you about how analogy results are usually reported?
3. Cosine similarity of `clinic` with `hospital`, with `pharmacy` and with `elephant`. Put them in order before you compute them.

For each result, write two things separately: **what the geometry returned**, and **your interpretation** of why.

a start

PYTHON

```
print(nearest(vector("doctor")))
analogy = vector("king") - vector("man") + vector("woman")
print("king - man + woman →", nearest(analogy, 4))
```

what it should print

OUTPUT

```
[('doctor', 1.0), ('nurse', 0.85), ('heal', 0.798), ('patient', 0.746), ('medicine', 0.646)]
king - man + woman → [('queen', 0.951), ('princess', 0.933), ('girl', 0.769), ('woman',
0.561)]
```